

Machine Learning for Forecasting Hazardous Air Quality Days in Indian Cities

Jaykumar Vinodbhai Chauhan

Student ID: 34710280

Master of Mathematics

MTH5000 Masters Research Project

Supervisor: Dr Tianhai Tian

School of Mathematics, Monash University

October 2, 2026

Abstract

Air pollution in Indian cities regularly reaches hazardous concentrations, and a forecast is useful only if it arrives before the hazardous day does. This project forecasts daily PM2.5 one to three days ahead at a Delhi monitor and evaluates the forecasts as a warning system rather than by average error alone. The central finding is that the alarm budget a decision maker accepts dominates the choice of model. Across ten specifications, from simple benchmarks and a classical ARIMA model with annual Fourier terms to four machine learning regressors and a recurrent network, the spread in hit rate at a fixed alarm budget is about five percentage points, while moving the budget from forty to one hundred alarms a year moves the hit rate by fifty. Machine learning earns its place only partially. The classical model beats every machine learning regressor at every horizon, a result strengthened rather than weakened by a sensitivity analysis on the cleaning threshold, and the one model that does win, a recurrent network, wins only at one day ahead, reported as a range across seeds rather than a single favourable one. Two real defects in the source monitor, a clipped instrument ceiling and a miscalculated coverage statistic, were diagnosed and corrected before modelling began. The recommendation is to design a warning system around its alarm budget, treating the forecasting model as a secondary refinement.

Contents

Abstract	i
1 Introduction	1
1.1 Research Background	1
1.2 Research Problem	1
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Research Significance	2
1.6 Structure of the Report	2
2 Literature Review	4
2.1 Air Quality Forecasting: Challenges and Characteristics	4
2.2 Classical and Machine Learning Methods for Time Series Forecasting	4
2.3 Deep Learning Methods for Time Series Forecasting	6
2.3.1 Recurrent Neural Networks and the Vanishing Gradient Problem	6
2.3.2 The Gated Recurrent Unit	6
2.4 Machine Learning Applications to Delhi and Indian Air Quality	7
2.5 Forecast Verification as a Warning System	8
2.6 Data Quality in Reference Air Quality Monitors	8
2.7 Research Gap and Chapter Summary	9
3 Methodology	10
3.1 Research Design	10
3.2 Data Collection and Station Selection	10
3.3 Data Preprocessing	11
3.3.1 Data Cleaning: Two Defects in the Source Data	11
3.3.2 Seasonality Analysis	12

3.3.3	Feature Construction	14
3.4	Machine Learning Methods	14
3.4.1	Classical Benchmark: ARIMA with Fourier Terms	14
3.4.2	Penalised Linear Regression	15
3.4.3	Random Forest	15
3.4.4	Histogram Gradient Boosting	16
3.5	Deep Learning Methods	17
3.5.1	The Gated Recurrent Unit: Architecture and Training	17
3.6	Rolling-Origin Evaluation	18
3.7	Evaluation Measures	18
3.8	Reproducibility	19
3.9	Chapter Summary	19
4	Results and Analysis	20
4.1	Forecast Accuracy	20
4.2	Two Arms of the Same Question	21
4.3	The Decision Threshold	21
4.4	Chapter Summary	23
5	Discussion	24
5.1	The Operational Question Dominates the Modelling Question	24
5.2	How Large Is a Difference Between Models?	24
5.3	Limitations	25
6	Conclusion	27
6.1	Summary of the Study	27
6.2	Main Conclusions	27
6.3	Future Work	28

List of Figures

3.1	The coverage defect. Panel (a) shows the number of readings contributing to each daily mean, with the cadence estimated from the data each year as the solid line and the fixed denominator of 24 as the dashed line. Panel (b) shows the rate of implausible values by true coverage.	12
3.2	The cleaned daily series, with days exceeding the threshold shaded. The episodic structure is the feature the project sets out to forecast: exceedances arrive as multi-day winter episodes rather than as isolated days.	12
3.3	No weekly cycle. Lags 7, 14 and 21 are circled and lie on the decay curve rather than above it. Every day of week interval contains zero.	13
3.4	The annual cycle, by contrast, is unmistakable: monthly median with the interquartile range shaded. Its strength is what makes the absence in Figure 3.3 credible rather than an artefact of a weak test.	13
4.1	The trade-off, by horizon. The top row shows hit rate against false alarm rate; the bottom row replaces the false alarm rate with alarms raised per year, the unit in which an operator budgets. The curves very nearly coincide, which is the finding. They have deliberately not been separated for legibility.	22

List of Tables

3.1	The analysis dataset after cleaning.	11
3.2	Cleaning rules applied, and the basis for each.	11
4.1	Relative mean absolute error against persistence, under rolling origin evaluation over 2,303 origins. Lower is better and 1.000 is persistence. The recurrent network is given as the range over five initialisation seeds; it is the only model whose result depends on one.	20
4.2	Best hit rate available within a budget of alarms per year, with the event fixed at the health threshold and only the decision threshold varying.	21

Chapter 1

Introduction

1.1 Research Background

Air pollution in Indian cities regularly reaches concentrations classified as hazardous to human health. Fine particulate matter small enough to be classified as PM_{2.5} penetrates deep into the respiratory system, and sustained epidemiological evidence links exposure to it with increased cardiopulmonary and all-cause mortality [Pope and Dockery, 2006, Chen and Hoek, 2020], which is the reason a health threshold rather than an arbitrary concentration is the event this project forecasts. The public response to such episodes depends on knowing that a severe day is coming before it arrives: a forecast issued on the morning of a hazardous day is of limited value, while one issued a day or more ahead can change behaviour, whether through health advisories, restrictions on outdoor activity, or school closures. Daily pollution series carry structure that makes such forecasting feasible in principle: persistence from one day to the next, a strong annual cycle driven by meteorology and seasonal agricultural burning, and dependence on recent local conditions.

My interest in this question is also personal. Having grown up in India, air pollution was not an abstract statistic but part of ordinary life: seasonal smog that turned daylight hazy, and days when going outside meant a sore throat. Whether those days can be predicted reliably enough to be worth warning people about is a question I have a genuine stake in.

1.2 Research Problem

Existing machine learning studies of Indian air quality (reviewed in Chapter 2) report forecast accuracy by average error, and do not decompose that accuracy by the operating point at which a warning would actually be raised. A model with respectable average error can still be a poor warning system if it systematically misses the extreme days that are the only ones anyone cares about, and the reverse also holds: two models with very different average error can produce nearly identical warning performance once a decision threshold is applied. Separately, the project this report was proposed under assumed that daily pollution series carry a weekly cycle alongside the annual one, driven by traffic and industrial activity, and treated reconciling two seasonal periods as a central modelling difficulty. Neither assumption is safe to carry into the modelling without testing it directly, and both are tested in this report.

1.3 Research Objectives

1. To develop and evaluate a suite of classical, machine learning, and deep learning models for forecasting daily PM_{2.5} concentrations one to three days ahead at a single, well-characterised Indian monitoring station, under an evaluation protocol that cannot leak future information into past forecasts.
2. To compare two formulations of the hazardous-day warning problem: forecasting the concentration and applying a health threshold to it, against classifying the exceedance event directly.
3. To quantify forecast skill as a warning system, using hit rate and false alarm rate under a varying decision threshold, rather than relying on average error alone, and to establish how much of that skill is attributable to the choice of model versus the choice of decision threshold.

1.4 Research Questions

1. How accurately can daily concentrations be forecast one to three days ahead by machine learning, relative to classical and naive benchmarks?
2. Is it better to forecast the concentration and then apply a threshold, or to classify the exceedance event directly?
3. Considered as a warning system, what hit rate and false alarm rate does each approach achieve, and how does the decision threshold trade them off?

The answer to the third question turns out to dominate the other two, and the report is organised accordingly.

1.5 Research Significance

The practical significance of this work is in the design of hazardous air quality warning systems: the finding that the decision threshold dominates the choice of model, developed in Chapter 4 and Chapter 5, implies that the alarm budget a health authority is willing to raise is a more consequential design decision than the forecasting model behind it. The methodological significance lies in applying a leakage-tested rolling-origin evaluation, rather than a single train-test split, to a model suite spanning classical statistics, machine learning, and a deep recurrent architecture, on a dataset small enough that the comparison between method families is itself informative about where each is reliable.

1.6 Structure of the Report

Chapter 2 reviews the literature on air quality forecasting, classical and machine learning methods for time series, deep learning methods for time series, and forecast verification as a warning system. Chapter 3 describes the data, the two defects found in it, and the full mathematical specification of every model

fitted, organised into machine learning methods and deep learning methods as the chapter's two central sections, followed by the rolling-origin evaluation protocol. Chapter 4 presents the results against the three research questions. Chapter 5 interprets those results and states the project's limitations. Chapter 6 concludes and proposes future work.

Chapter 2

Literature Review

2.1 Air Quality Forecasting: Challenges and Characteristics

Daily particulate matter series present a specific set of statistical challenges that distinguish them from the time series most forecasting methods were developed against. The distribution of PM_{2.5} concentrations is strongly right skewed, with a small number of extreme episodes, rather than normally distributed around a stable mean; the series is strongly seasonal at the annual scale; and the quantity of practical interest, whether a day exceeds a health threshold, is a minority class, typically under a third of days even at a severe site. These three properties motivate working in logarithms rather than on the raw concentration scale (Section 3.4.1), motivate testing rather than assuming the seasonal structure present in the series (Section 3.3.2), and motivate evaluating forecasts as a warning system rather than by a single average-error statistic (Section 2.5).

2.2 Classical and Machine Learning Methods for Time Series Forecasting

The classical approach to a series with this structure is an autoregressive integrated moving average (ARIMA) model, in which a series is differenced to stationarity and then modelled as a linear combination of its own past values and past forecast errors, following the identification and estimation methodology set out by Box et al. [2008]. Seasonal variants extend this by one additional seasonal period, but a series with two seasonal periods, for instance a weekly cycle and an annual one, cannot be accommodated by a single seasonal ARIMA order, which is one of the two assumptions this project tests directly rather than carrying over unexamined. The order itself is conventionally chosen by an information-criterion search over a bounded grid rather than fixed in advance, an approach formalised for routine use by Hyndman and Khandakar [2008]; this project follows the same principle, an explicit grid scored by the Akaike information criterion rather than a single order asserted without justification (Section 3.4.1). Where the seasonal pattern itself is complex, for instance more than one period or a period that does not divide the sampling interval evenly, Fourier terms offer a compact way to carry it as a small number of exogenous regressors rather than as a large seasonal order; De Livera et al. [2011] develop this idea within an exponential smoothing framework for series with multiple seasonal periods, and the same device, harmonic regressors at a fixed set of frequencies, is the one this project uses to carry the annual

cycle in an ARIMA model’s exogenous term (Section 3.4.1).

Machine learning offers an alternative that does not assume a linear relationship between a forecast and its predictors. Penalised linear regression controls the variance of an ordinary least squares fit by shrinking its coefficients, which matters when predictors are highly correlated, as lagged and rolling-window features of the same series necessarily are [Hoerl and Kennard, 1970]. Random forests and gradient boosting both build on classification and regression trees [Breiman et al., 2017], which recursively partition the feature space to minimise within-region error, but combine many trees by two different mechanisms. A random forest averages trees fitted independently to bootstrap-resampled data, a technique named bootstrap aggregating, or bagging, and shown to reduce the variance of an unstable predictor such as a tree without changing its bias [Breiman, 1996, 2001]. Gradient boosting instead fits an ensemble additively, each new tree correcting the residual error of the ensemble so far; this traces to boosting algorithms that combine many weak learners into a strong one by reweighting misclassified examples [Freund and Schapire, 1997], which Friedman [2001] reformulated as stagewise functional gradient descent, the form implemented by the histogram-based estimator used in this project (Section 3.4) and, in a widely used alternative implementation not used here, by XGBOOST [Chen and Guestrin, 2016]. None of these three machine learning methods assumes stationarity, a seasonal order, or a linear functional form, which is their appeal relative to ARIMA; their cost is that they fit the function empirically from a necessarily finite sample, and Makridakis et al. [2020] report, across 100,000 series in the M4 forecasting competition, that this flexibility does not reliably translate into an advantage over classical benchmarks on series of a few thousand observations, which is the order of magnitude available in this project. This was not a new finding when M4 confirmed it at scale: Makridakis et al. [2018] had already reported the same pattern on the smaller M3 dataset and argued that the forecasting community had adopted machine learning methods faster than the evidence for their superiority had accumulated, and Siarni-Namini et al. [2018] report a direct head-to-head instance of it, an LSTM failing to beat ARIMA on a set of financial and economic series very different in domain from this project’s but identical in being short relative to a deep network’s capacity. The picture is not uniformly negative for machine learning, however: in the M5 competition, run on retail demand series with genuine exogenous and hierarchical structure to exploit, gradient boosting methods won decisively [Makridakis et al., 2022]. Read together, M4 and M5 suggest the deciding factor is not machine learning versus classical methods in the abstract but whether the series on offer gives a flexible method something structural to exploit beyond what a classical model already captures; Chapter 4 revisits this reading directly; a single daily PM2.5 series with no exogenous predictors is closer to the M4 setting than the M5 one, which is the comparison this project’s own result in Chapter 4 confirms rather than contradicts. A different strand of this literature asks not which family wins outright but whether they can be combined: Zhang [2003] fit a neural network to an ARIMA model’s residuals and report a gain over either model alone. This project does not pursue a hybrid, evaluating the two families side by side instead, precisely so that the comparison in Chapter 4 is informative about which family suits this series rather than obscured by a blend of both.

Whichever family is used, the comparison is only as trustworthy as the procedure used to test it, a methodological point this project treats as seriously as the choice of model. Tashman [2000] reviews decades of out-of-sample forecasting evaluations and argues that a single train-test split, still common practice at the time, understates how much accuracy varies across time periods; Hewamalage et al. [2023] revisit the same concern two decades later and document the same pitfalls still occurring routinely in

applied forecasting work, including exactly the single-split optimism this project found and corrected in Section 3.6. The rolling-origin design adopted here (Section 3.6) is this project’s answer to both papers’ shared complaint. The accuracy measure itself is a related choice: Hyndman and Koehler [2006] propose the mean absolute scaled error specifically because percentage errors are undefined when a series can be zero and unstable when it is close to zero, and this project reports it alongside the relative mean absolute error introduced in Section 3.7 for exactly the reason their paper gives, that a scaled error’s apparent improvement can reflect a change in the series’ own volatility rather than a change in forecast skill, which is precisely what was found when the single split was replaced by the rolling-origin harness.

2.3 Deep Learning Methods for Time Series Forecasting

2.3.1 Recurrent Neural Networks and the Vanishing Gradient Problem

A recurrent neural network processes a sequence by maintaining a hidden state that is updated at every timestep as a function of the current input and the previous hidden state, which in principle lets it represent dependence on the arbitrarily distant past without the fixed lag structure a regression feature table imposes, trained by backpropagation through time, a direct extension of the backpropagation algorithm for feedforward networks [Rumelhart et al., 1986]. In practice, a plain recurrent network trained this way suffers from vanishing or exploding gradients as the sequence lengthens: Bengio et al. [1994] show formally that the conditions which let a recurrent network store information robustly over many timesteps are, under gradient descent, precisely the conditions under which the gradient of a distant error with respect to an early input either vanishes or explodes, so that reliably learning long-range dependence and reliably training by gradient descent are in tension with each other. Hochreiter and Schmidhuber [1997] address this with the long short-term memory (LSTM) architecture, a memory cell and three multiplicative gates controlling what is written to, read from, and forgotten by that cell, at the cost of a comparatively large number of parameters for a short daily series.

2.3.2 The Gated Recurrent Unit

The gated recurrent unit (GRU) simplifies the LSTM to two gates and no separate memory cell, while retaining the property that gradients can flow across many timesteps without vanishing [Cho et al., 2014]. Its full specification, and the specific architecture used in this project, is given in Section 3.5.1. The GRU’s appeal for a daily series with roughly three thousand observations is precisely its smaller parameter count relative to an LSTM, which matters more on a dataset this size than the slightly larger representational capacity an LSTM’s extra gate and cell state provide. Lim and Zohren [2021] survey the broader move of deep learning into time series forecasting of which the GRU and LSTM are one part, alongside convolutional and attention-based sequence architectures developed since, and note that the gain such architectures offer over classical methods depends heavily on how much data and exogenous structure a given forecasting problem provides, the same conditioning factor Section 2.2 identifies as deciding whether machine learning or a classical benchmark wins on a series of a given size; this survey, and the general account of deep learning’s strengths and limits given by LeCun et al. [2015], frame the GRU result in Chapter 4 as an instance of a known pattern rather than an isolated one. A

practical limitation specific to recurrent architectures, and directly relevant to this project’s own series, is that they require a value at every timestep and cannot represent a missing observation natively; Che et al. [2018] study exactly this problem for multivariate clinical time series and adopt the same device used in Section 3.5.1, carrying the last observed value forward alongside an explicit missingness indicator, as a principled rather than ad hoc way to hand a recurrent network a gap in its input.

2.4 Machine Learning Applications to Delhi and Indian Air Quality

Delhi’s air pollution problem is well characterised at the level of sources and policy before any forecasting model is applied to it. Guttikunda et al. [2014] review the emission inventory across major Indian cities and identify vehicular traffic, industry, and seasonal biomass burning as the dominant contributors, the last of which is the specific mechanism behind the stubble burning season discussed below. Satellite-based studies independently quantify that mechanism’s contribution to Delhi’s severe episodes: Cusworth et al. [2018] attribute a substantial share of Delhi’s post-monsoon particulate spikes to agricultural fires further north, and Jethva et al. [2019] connect the same fires to reduced regional air quality using more than a decade of satellite fire counts. None of these three studies forecasts the series; they establish the physical mechanism that a purely statistical model, such as every model in this project, can only infer indirectly from the series’ own seasonal structure, which is worth stating plainly as a limitation of the approach taken here.

Machine learning applied specifically to Delhi PM_{2.5} is not a new proposal. Masood and Ahmad [2020] fit two models, a support vector machine and an artificial neural network, to predict Delhi’s PM_{2.5} concentrations from meteorological and pollutant covariates and report that the neural network is the more accurate of the two; their inputs are exogenous measurements at the forecast target day itself rather than the series’ own lagged history, which is a different forecasting problem from the one addressed here, where the only inputs available at origin are the series’ own past. Singh and Srivastava [2025] narrow the problem to Delhi’s autumn stubble burning season, when biomass burning in neighbouring states drives the city’s most severe episodes and, by their own account, its air pollutant levels peak in October and November specifically. Their paper combines two analyses rather than one: a health-risk assessment of stubble-season exposure using a dedicated health-risk model, and, separately, a comparison of three machine learning models, an artificial neural network, a support vector machine, and a random forest, for PM_{2.5} forecasting, with the random forest the most consistent of the three. The health-risk analysis and the forecasting comparison sit alongside each other rather than one inside the other: the forecasting models are themselves still scored by the same average-error measures, R^2 , RMSE and MAE, that this project argues are insufficient on their own, not by a health-weighted criterion. Neither paper evaluates its forecasts as a warning system in the sense used here, with an explicit alarm budget and a false alarm rate reported alongside the hit rate; both report accuracy on held-out data without decomposing it by the operating point at which an alarm would actually be raised. That gap, between forecast accuracy and warning system performance, is the space this project’s third research question occupies.

The finding that a classical benchmark beats every machine learning model in this project’s own results (Chapter 4) is not an anomaly against the wider forecasting literature discussed in Section 2.2. The mechanism Makridakis et al. [2020] propose, that a few thousand observations is a small sample for

a flexible learner relative to the number of parameters a modern architecture can fit, applies directly to a project with roughly three thousand daily observations. The negative result for machine learning at two and three days ahead in Chapter 4 is therefore the expected outcome under that literature rather than a surprise specific to this dataset.

2.5 Forecast Verification as a Warning System

The distinction this project draws between forecast accuracy and warning system performance, hit rate, false alarm rate, and the trade-off between them as the decision threshold moves, follows the verification framework standard in operational meteorology [Jolliffe and Stephenson, 2012]. That literature developed around exactly the situation faced here: a forecast of a rare, high-impact event, where the average error of a model is a poor guide to its value as a warning, and where a contingency table decomposed by decision threshold is more informative than any single accuracy score. Its central methodological point, that verification measures should be chosen to match the decision a forecast supports rather than borrowed by default from a different problem, motivated reporting hit rate and false alarm rate as a pair in Section 3.7 rather than a single skill score. The critical success index reported alongside them, the fraction of forecast or observed events that were correctly forecast, with correct quiet days excluded from both numerator and denominator, was introduced by Schaefer [1990] specifically to avoid a statistic inflated by the large number of quiet days that dominate a rare-event problem such as this one, where fewer than a third of days exceed the health threshold.

That same imbalance, a minority of hazardous days against a majority of quiet ones, is the reason this project never reports plain classification accuracy as a warning-system measure. He and Garcia [2009] survey the broader problem of learning from imbalanced data and document how a classifier can score highly on overall accuracy by predicting the majority class essentially always, a failure mode this project’s own alarm-budget framing (Section 3.7) is designed to make visible rather than hide, since an alarm budget of zero would trivially achieve a hit rate of zero rather than a flattering accuracy figure. Saito and Rehmsmeier [2015] make the related point that a receiver operating characteristic curve, a common default for a binary classifier, can look similarly optimistic on imbalanced data because it is insensitive to a large true-negative count that costs nothing to accumulate; a precision-recall-style decomposition, of which the hit rate and false alarm rate pair used throughout this report is a close relative, is the more informative choice for exactly this kind of problem.

2.6 Data Quality in Reference Air Quality Monitors

The data quality problems documented in Section 3.3.1 are not unique to this station. Jiang et al. [2023] address the companion problem to the one found here: reference monitors occasionally report physically impossible negative PM_{2.5} concentrations, arising from instrument noise near zero rather than a true measurement, and propose a defensible statistical threshold below which a reading should be treated as invalid rather than retained or naively clipped to zero. Their approach and the one used in Section 3.3.1 for the clipped ceiling at 1990 share a method rather than a target: both treat an implausible value as a data quality question to be settled by evidence, specifically its relationship to the plausible range and its

recurrence pattern, rather than by a rule of thumb applied uniformly. Considered together, artefacts at both ends of the distribution suggest that reference-grade air quality monitors carry systematic measurement pathologies that a forecasting study must characterise explicitly rather than assume away.

A third data quality question, separate from an implausible value being present, is a plausible value being absent. Little and Rubin [2019] set out the standard framework for reasoning about why data are missing, missing completely at random, missing at random given other observed information, or missing in a way that depends on the unobserved value itself, and show that the appropriate treatment differs sharply between the three. The coverage rule adopted in Section 3.3.1, excluding a day when too few readings contributed to its mean, is a judgement that the gaps here are closer to the first two categories than the third: nothing in the station’s own account of why a reading is missing suggests that a reading is more likely to be dropped specifically when the true concentration is unusually high or low, as opposed to for reasons such as instrument maintenance that are unrelated to the pollution level itself. The remaining short gaps that survive the coverage rule are carried forward rather than discarded, using the same forward-fill-plus-indicator treatment discussed for the recurrent network in Section 3.5.1, which is defensible under the same assumption.

2.7 Research Gap and Chapter Summary

This chapter has covered five bodies of literature: the statistical characteristics of air quality series that motivate the methodological choices in Chapter 3; classical and machine learning methods for time series forecasting, their shared lineage through bagging, boosting and regression trees, and why the forecasting-competition literature predicts a flexible learner need not beat a classical benchmark on a small sample, together with the methodology for testing that comparison honestly, out-of-sample evaluation and scale-free accuracy measures, rather than taking any single reported number at face value; deep learning methods for sequences, the specific architectural reason a GRU rather than an LSTM suits a dataset this size, and the structural limitation, an inability to represent a missing input, that a recurrent architecture carries regardless of which gated variant is chosen; the physical and epidemiological background, emission sources, biomass burning, and the health consequences of exposure, that motivates treating this as a hazard-warning problem rather than a curve-fitting exercise; and the two literatures, forecast verification and air quality monitor data quality, that this project’s own findings connect back to directly. Taken together, this literature places the project’s contribution precisely. Applying an established model suite to Delhi’s air quality problem is not new, and neither is the finding that classical methods hold their ground against machine learning on a modest sample. What the Delhi-specific studies above do not do is evaluate that comparison as a warning system with an explicit, varying alarm budget, using verification methodology from a literature that reaches air quality forecasting only indirectly. Chapter 4 shows why that omission matters: the ranking of models looks different once accuracy is separated from operational usefulness, a finding neither literature taken alone would have anticipated.

Chapter 3

Methodology

3.1 Research Design

This chapter proceeds from data to model to evaluation. Section 3.2 describes the data source and the station-selection process. Section 3.3 describes two defects found in the raw series, the seasonality analysis that overturned an assumption in the approved proposal, and the feature table built from the cleaned series. Section 3.4 and Section 3.5 give the full mathematical specification of the two families of model this project compares, machine learning and deep learning, each covering the method’s general form before the specific configuration used here. Section 3.6 specifies the rolling-origin evaluation every model is scored under, and Section 3.7 specifies the accuracy and warning-system measures reported in Chapter 4.

3.2 Data Collection and Station Selection

Daily mean $\text{PM}_{2.5}$ concentrations were obtained from OpenAQ. Five candidate monitoring stations within 25 km of central Delhi were compared on data coverage rather than on nominal length of record.

The comparison produced an unexpected constraint. Every station operated by the Central Pollution Control Board returns measurements only from 19 February 2025, approximately eighteen months, although the OpenAQ location metadata advertises a first observation in February 2016. Four independent stations sharing an identical cutoff indicates a systematic limit on what the measurements endpoint serves rather than a coincidence. Only location 8118 returns the full record and it was therefore selected.

This carries a limitation that must be stated plainly. Location 8118 is supplied through AirNow and is a United States diplomatic post monitor rather than part of the Indian official network. The project therefore forecasts at a single well-characterised reference site in Delhi, which is common in this literature, and the results should not be read as representative of the city as a whole.

Table 3.1: The analysis dataset after cleaning.

Source	OpenAQ location 8118, sensor 23534, provider AirNow
Period	2016-11-09 to 2026-08-27
Calendar days	3579
Usable daily means	3093 (86.4% of calendar)
Longest gap	34 consecutive days
Mean, median, maximum	102.7, 71.7, 896.0
Days above 121	920 (29.7%)

3.3 Data Preprocessing

3.3.1 Data Cleaning: Two Defects in the Source Data

Two problems were identified before any modelling, one of which was silently corrupting the series.

Physically impossible values. The raw series contained five negative concentrations and a maximum of $1990\text{ }\mu\text{g m}^{-3}$. The value 1990.0 appeared three times identically on unrelated dates, which is the signature of a clipped instrument ceiling rather than a measurement. One occurrence fell on 2 September, during the monsoon, between neighbouring days of 42 and 178.

An incorrect coverage denominator. OpenAQ reports the completeness of each daily aggregate as the observation count divided by an expected count of 24, assuming hourly reporting. The sensor at location 8118 reported half-hourly from 2016 to 2024 and hourly from 2025 onward. The visible symptom is a reported completeness exceeding 100 per cent, which is impossible. The damaging symptom is invisible: a day holding 24 of a genuine 48 readings is scored as fully complete, so a mean computed from half a day of a polluted afternoon enters the series as though it were a daily mean. Days at 50 to 75 per cent true coverage produced implausible values approximately seven times as often as days above 90 per cent, which identifies incomplete days as the mechanism rather than a coincidence.

The correction takes the reporting cadence from the data, using the ninetieth percentile of observation counts within each year, which follows the 2025 change in reporting interval automatically. Recomputed on that basis, 1,085 of the 3,467 raw calendar-linked days fall below 75 per cent true coverage, far more than the 268 days that have no recorded value at all and would be the only ones flagged by a check for whether a value exists rather than how reliably it was measured.

Table 3.2: Cleaning rules applied, and the basis for each.

Rule	Days removed	Basis
Value below zero	5	Mass concentration cannot be negative
Coverage below 50%	270	Against the cadence taken from the data
Value above 1000	7	Judgement; sensitivity analysis in Section 5.3
Total	282 of 3,375	

The coverage threshold is set at 50 per cent rather than a stricter value because the longest gap is the

binding constraint: at 60 per cent the worst gap rises to 69 days and at 90 per cent to 185 days, either of which would distort estimation of the annual cycle. At 50 per cent the worst gap is 34 days. The exceedance rate moves only from 30.0 to 27.0 per cent across every candidate threshold, which confirms that the filter is not selectively removing the events of interest.

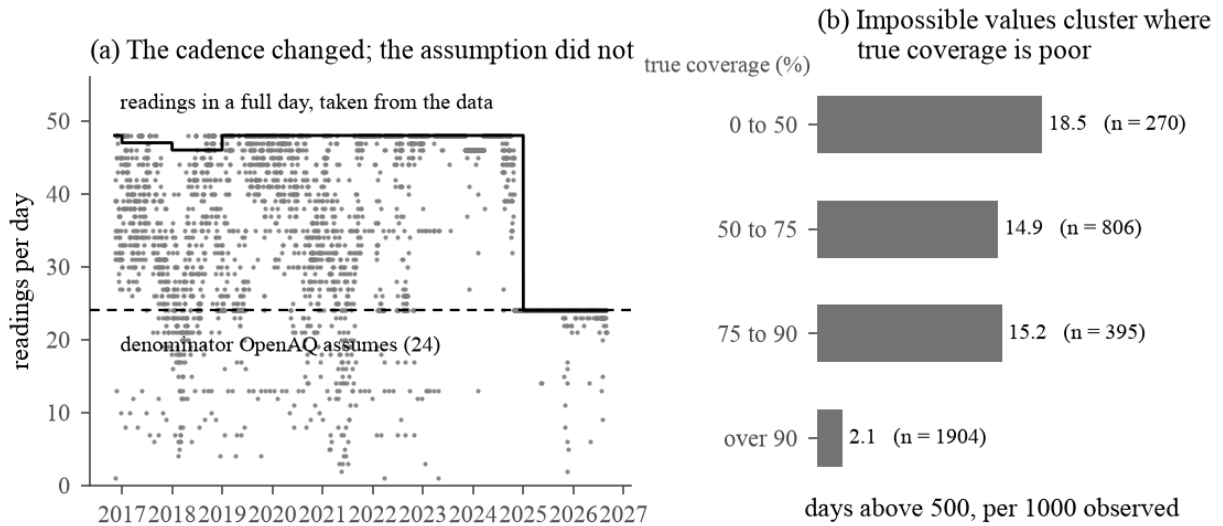


Figure 3.1: The coverage defect. Panel (a) shows the number of readings contributing to each daily mean, with the cadence estimated from the data each year as the solid line and the fixed denominator of 24 as the dashed line. Panel (b) shows the rate of implausible values by true coverage.

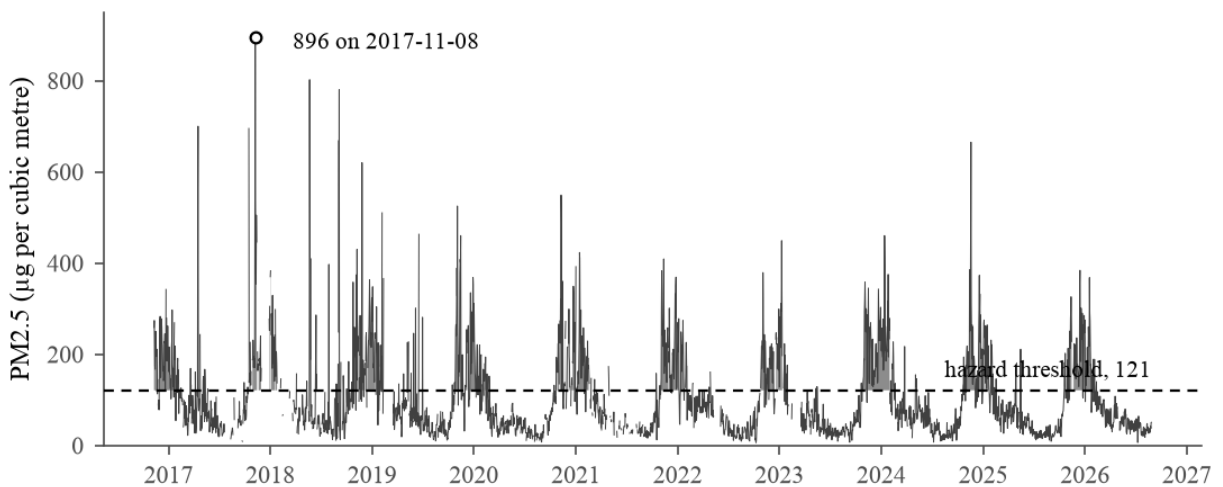


Figure 3.2: The cleaned daily series, with days exceeding the threshold shaded. The episodic structure is the feature the project sets out to forecast: exceedances arrive as multi-day winter episodes rather than as isolated days.

3.3.2 Seasonality Analysis

The approved proposal states that daily pollution series carry a weekly cycle driven by traffic and industrial activity, and treats the coexistence of a weekly and an annual period as the central modelling difficulty, since a seasonal ARIMA accommodates only one seasonal period.

Tested on the cleaned series, the weekly cycle is not present at this station. Three independent tests agree. The periodogram gives dominant periods of 357.9, 178.9, 188.4, 397.7, 325.4 and 60.7 days,

which is the annual cycle and its harmonics, with nothing near seven. On the deseasonalised logarithm of concentration the autocorrelation at lag 7 is 0.173, while lag 6 is 0.179 and lag 8 is 0.153, so lag 7 lies on the decay curve rather than above it; the same holds at lags 14 and 21. The day of week effect on the same residuals spans 6.5 per cent between the highest and lowest day, with an analysis of variance giving $F = 1.13$, $p = 0.34$ and a Kruskal-Wallis test giving $p = 0.77$.

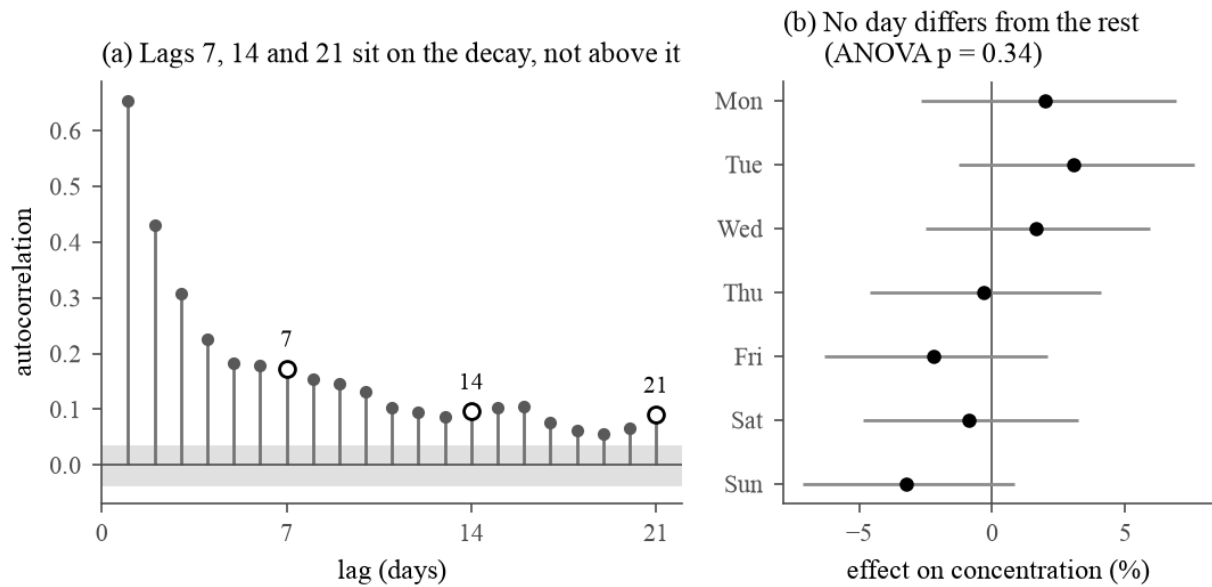


Figure 3.3: No weekly cycle. Lags 7, 14 and 21 are circled and lie on the decay curve rather than above it. Every day of week interval contains zero.

There is a physical reading rather than a null result. A weekly rhythm in urban particulate concentration is a traffic signature, and location 8118 sits in the diplomatic enclave rather than at a roadside site such as ITO or Anand Vihar. Its absence at a site that is not traffic dominated is what one would expect.

The consequence for the specification is direct: the classical benchmark is ARIMA with annual Fourier terms as exogenous regressors and no weekly seasonal order. The specification is simpler, better identified, and empirically justified rather than assumed.

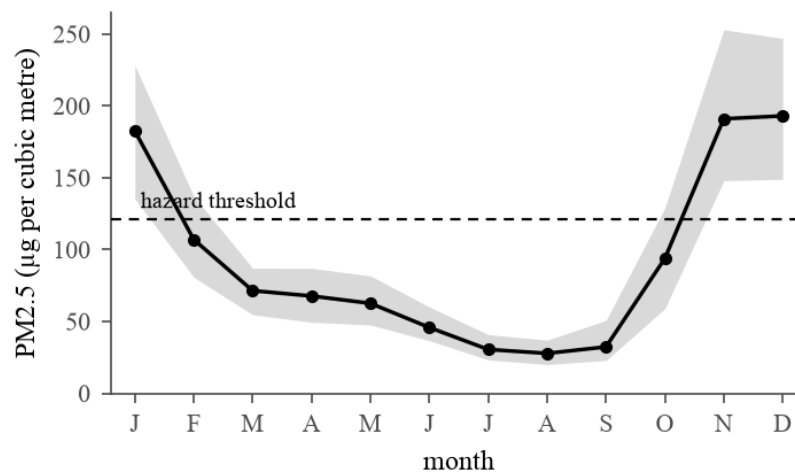


Figure 3.4: The annual cycle, by contrast, is unmistakable: monthly median with the interquartile range shaded. Its strength is what makes the absence in Figure 3.3 credible rather than an artefact of a weak test.

3.3.3 Feature Construction

A supervised table was built with one row per forecast origin. A row indexed by date t carries only functions of observations up to and including day t , and its targets are the concentrations on days $t + 1$, $t + 2$ and $t + 3$. Features comprise lags at 0, 1, 2, 3 and 7 days; first differences; rolling mean, standard deviation and maximum over 3, 7, 14 and 30 days; an anomaly and standardised anomaly against the 30 day norm; exceedance frequency over the last 7 and 30 days; days since the last exceedance; calendar terms; four pairs of annual Fourier terms; and three missingness indicators.

Missing values are never imputed. Gradient boosting accepts them natively and the state space models skip the measurement update at a gap, so the absence is carried as information rather than replaced by a fabricated value. The single exception is the recurrent network, discussed in Section 3.5.1.

3.4 Machine Learning Methods

Six of the ten specifications fitted in this project belong to the machine learning family in the broad sense used in Section 2.2: the classical ARIMA benchmark, which is included here for comparison even though it predates the term, and the three explicitly machine learning regressors, each fitted to both the level of concentration and its change from the forecast origin. Every model in this section is fitted to the logarithm of concentration rather than the raw scale. Annual Fourier terms explain 69 per cent of the variance of the logarithm against considerably less on the raw scale, and fitting in logs prevents a model predicting a negative concentration, which has no physical meaning.

3.4.1 Classical Benchmark: ARIMA with Fourier Terms

Let $y_t = \log(\text{PM2.5}_t)$ and let B denote the backshift operator, $By_t = y_{t-1}$. Following the Box-Jenkins formulation [Box et al., 2008], an autoregressive integrated moving average model of order (p, d, q) with exogenous regressors x_t is

$$\phi(B)(1-B)^d y_t = c + \theta(B)\varepsilon_t + \beta^\top x_t, \quad (3.1)$$

where $\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ is the autoregressive polynomial, $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$ is the moving average polynomial, and ε_t is white noise. The order selected from an explicit grid by the Akaike information criterion on the training block is $(p, d, q) = (1, 1, 1)$, so Equation 3.1 specialises to

$$(1 - \phi_1 B)(1 - B)y_t = c + (1 + \theta_1 B)\varepsilon_t + \sum_{k=1}^4 \left[\alpha_k \sin\left(\frac{2\pi kt}{365.25}\right) + \beta_k \cos\left(\frac{2\pi kt}{365.25}\right) \right], \quad (3.2)$$

with four pairs of annual Fourier terms ($K = 4$) as the exogenous regressors x_t , carrying the annual cycle established in Section 3.3.2 and deliberately carrying no weekly seasonal order. Using trigonometric regressors at a fixed set of frequencies to carry a seasonal pattern inside an otherwise standard model, rather than through a seasonal differencing order, follows the same logic De Livera et al. [2011] apply to exponential smoothing for series with seasonal patterns too complex for a single seasonal order to represent; here the device is used to keep the annual cycle outside the ARIMA order entirely, which is what makes carrying no weekly term in parallel straightforward rather than a second seasonal order

fighting the first.

Back-transformation from the log scale is not innocent. If $\log y$ has forecast mean m and forecast variance s^2 , then under the model's Gaussian error assumption y is lognormally distributed, and

$$\text{median}(y) = \exp(m), \quad \text{mean}(y) = \exp\left(m + \frac{1}{2}s^2\right). \quad (3.3)$$

Both quantities were produced as separately named models, since a report that back-transforms without stating which one it reports has silently changed what is being forecast. The gap between the two, and specifically the tendency of the median variant to systematically under-forecast a right-skewed quantity, is a known bias rather than a property special to this project: Miller [1984] derive the same correction term, $\exp(\frac{1}{2}s^2)$ in Equation 3.3, for curve-fitting contexts generally, as the adjustment needed to recover an unbiased mean from a model fitted on the logarithm of a quantity.

3.4.2 Penalised Linear Regression

Given the feature matrix $X \in \mathbb{R}^{n \times p}$ built in Section 3.3.3 and the target vector y , ordinary least squares is unstable when predictors are highly correlated, as lagged and rolling-window features of the same series necessarily are. Ridge regression [Hoerl and Kennard, 1970] controls this by penalising the size of the coefficient vector:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad (3.4)$$

with closed-form solution $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$. The penalty $\lambda > 0$ shrinks the coefficients toward zero, trading a small increase in bias for a reduction in variance; λ is selected by cross-validation on the training block at every refit.

3.4.3 Random Forest

A random forest [Breiman, 2001] fits B regression trees to independent bootstrap resamples of the training data, each tree considering only a random subset of features at every split, and averages their predictions. Averaging over bootstrap resamples in this way is bootstrap aggregating, or bagging [Breiman, 1996]; a random forest adds the random feature subset at each split on top of bagging specifically to decorrelate the resulting trees further, since trees grown on overlapping bootstrap samples of the same series would otherwise tend to split on the same dominant lag features and average out less variance than independent trees would:

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x). \quad (3.5)$$

Each tree T_b is grown by recursively partitioning the feature space: at a node with data $\{(x_i, y_i)\}$, the split (j, s) on feature j at threshold s is chosen to minimise the total within-region sum of squared error,

$$\min_{j,s} \left[\sum_{i: x_{ij} \leq s} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_{ij} > s} (y_i - \bar{y}_{R_2})^2 \right], \quad (3.6)$$

where R_1 and R_2 are the two resulting regions and $\bar{y}_{R_1}, \bar{y}_{R_2}$ their mean targets, the splitting rule set out for classification and regression trees generally by Breiman et al. [2017]. Averaging over many such trees, each trained on a different resample and a different random subset of features at each split, reduces the variance of a single tree’s prediction without imposing a linear functional form.

3.4.4 Histogram Gradient Boosting

Gradient boosting [Friedman, 2001] fits an ensemble additively rather than by averaging independent trees, generalising an older idea, boosting, in which many weak learners are combined into one strong learner by repeatedly reweighting the examples a classifier got wrong so that the next learner in the sequence focuses on them [Freund and Schapire, 1997]. Friedman’s reformulation recasts that reweighting as gradient descent in function space rather than in the space of example weights, which is what lets the same framework cover arbitrary differentiable loss functions rather than only classification error. The ensemble is initialised as a constant, $F_0(x) = \arg \min_{\gamma} \sum_i L(y_i, \gamma)$, and for $m = 1, \dots, M$ a regression tree h_m is fitted to the negative gradient of the loss with respect to the current ensemble’s predictions,

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}, \quad (3.7)$$

which for squared-error loss reduces to the ordinary residual $r_{im} = y_i - F_{m-1}(x_i)$, and the ensemble is updated by

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad (3.8)$$

where $0 < \nu \leq 1$ is a shrinkage (learning rate) parameter that controls how much each new tree is allowed to correct the ensemble. The histogram-based implementation used here bins continuous features before searching for splits, which accepts missing values natively, consistent with the decision in Section 3.3.3 never to impute. This is the same binning strategy that made gradient boosting practical on large tabular datasets in implementations such as XGBOOST [Chen and Guestrin, 2016]; scikit-learn’s histogram-based estimator used here applies the same idea at the scale this project needs, a few thousand rows rather than millions, so the speed advantage that motivated the original design matters less here than the fact that it accepts missing values without a separate imputation step. Both the number of boosting iterations M and the per-tree leaf count were chosen at every refit from a small explicit grid, since an uncontrolled M overfits badly on a series of this length, a failure mode encountered and corrected during development and reported in Section 5.3.

Both the random forest and the gradient boosting models were fitted twice, once to the level of log concentration and once to its change from the forecast origin, since a tree cannot extrapolate beyond the range of targets it was trained on and this series drifts across the decade of record; modelling the change removes that handicap by asking the tree to represent a roughly stationary residual rather than a trending level.

3.5 Deep Learning Methods

3.5.1 The Gated Recurrent Unit: Architecture and Training

The gated recurrent unit [Cho et al., 2014] maintains a hidden state $h_t \in \mathbb{R}^H$ updated at each timestep t by two gates. The update gate z_t controls how much of the previous state is retained against how much is replaced by a new candidate state; the reset gate r_t controls how much of the previous state is used in forming that candidate:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (3.9)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad (3.10)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \quad (3.11)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad (3.12)$$

where σ is the logistic sigmoid, $\odot$ is elementwise multiplication, and $W_z, W_r, W_h, U_z, U_r, U_h, b_z, b_r, b_h$ are the parameters learned by training, fitted by backpropagation through time, the sequential application of the backpropagation algorithm [Rumelhart et al., 1986] across every timestep in the input window. Because the final state h_t in Equation 3.12 is a convex combination of the previous state and the new candidate rather than a product of many successive nonlinear transformations, gradients can flow backward through many timesteps without vanishing in the sense formalised by Bengio et al. [1994], which is the property that motivated gated architectures in Section 2.3.

The specific network fitted in this project is a single GRU layer with $H = 24$ hidden units, reading a sequence of the last 30 days ($t - 29, \dots, t$), each day carrying three channels: the logarithm of concentration, a missingness flag, and the annual Fourier terms from Section 3.3.3. A linear output layer maps the final hidden state h_t to a single predicted value, the change in log concentration from the forecast origin, with the origin value added back afterwards; modelling the change rather than the level is the same device used for the tree models in Section 3.4, for the same reason. The network is trained by minimising mean absolute error with the Adam optimiser, with early stopping on a chronological holdout, the most recent 15 per cent of each training window, so that training stops once validation error no longer improves rather than at a fixed epoch count.

A recurrent network forces a compromise that no other model in this project requires. It must be handed a value at every timestep: it cannot skip one the way the state space model in Section 3.4.1 skips a measurement update, and it cannot accept a missing value the way the histogram gradient boosting model in Section 3.4 can. Gaps are therefore carried forward in the log-concentration channel, with the separate missingness flag marking which days were filled rather than observed, the same forward-fill-plus-indicator construction Che et al. [2018] use for recurrent networks over clinical time series with the same structural gap, a value required at every timestep and a sensor that does not always provide one. The one architecture Dr Tian specifically asked to be included is the one architecture in the suite that cannot represent the statement that a value is unknown.

Because the network's weights are initialised randomly, its forecast depends on that initialisation, unlike every other model in this project. It is therefore fitted independently across five random seeds, and its result is reported throughout Chapter 4 as the range across those five runs, never as a single seed's

point estimate; the justification for treating this as a reporting requirement rather than a presentational choice is given in Section 4.1.

3.6 Rolling-Origin Evaluation

Every model was evaluated over 2,303 origins from 9 November 2019 to 24 August 2026, spanning 6.8 years, with a three year burn-in, parameters re-estimated every 90 days and model state advanced daily in between.

This is not a refinement of a single train-test split; it replaces one, for the reason standard cross-validation is inappropriate for dependent data in the first place: shuffling origins lets a model trained on the future score itself against the past [Bergmeir and Benítez, 2012]. On a single test block the ARIMA scored 0.903 in relative terms at one day ahead; under the rolling-origin harness built to replace it, the same model scores 0.940, worse rather than better, because the single block happened to be calmer than the training period and was flattering every model. A single split is one draw from a process, and a conclusion drawn from it is a conclusion about that period, exactly the instability in single-split evaluation that Tashman [2000] documents across decades of applied forecasting studies and that Hewamalage et al. [2023] report as still common practice. The rolling-origin design used throughout this project is adopted specifically to avoid being another instance of the pattern both papers describe.

The governing invariant is that at origin t a model may use observations up to and including day t and nothing else: not the value at $t + 1$, not a parameter estimated using data after t , and not a scaling constant or imputed value computed from the full series. Since nothing fails visibly when that is broken, and the numbers merely improve, the invariant is enforced by construction and tested directly. Every model runs through one shared loop, and that loop is verified by corrupting the series from a chosen date, rerunning, and requiring that no forecast issued before that date changes.

One trap is specific to supervised learning under a rolling origin. At origin t the label attached to feature row t' is the concentration at $t' + h$, so the training set ends at $t - h$ rather than at t . A separate model is fitted per horizon because that arithmetic requires it. The error would have amounted to three rows per refit and would have been entirely invisible in the output.

3.7 Evaluation Measures

Accuracy is reported separately at each horizon and never averaged, since skill decays with horizon and concealing that would defeat the purpose. The headline measure is relative mean absolute error against the persistence forecast computed on identical rows, so that 1.000 is persistence and values below one beat it. Mean absolute scaled error, proposed by Hyndman and Koehler [2006] as a scale-free accuracy measure that remains defined even when the series passes through values near zero, is reported alongside, but it divides by the training period volatility and therefore also reflects any difference in volatility between periods, which is material here; this is the same distinction between a scaled error and a relative error that motivated reporting both rather than either alone.

As a warning system, a day is an event if the observed concentration exceeds the health threshold,

and an alarm is raised when the forecast exceeds a decision threshold. These are separate quantities and are kept separate throughout. Reported are the hit rate, the false alarm rate defined as false alarms divided by quiet days, precision, the critical success index [Schaefer, 1990], and alarms raised per year. The last is included because a false alarm rate of 0.10 sounds negligible and means roughly one wasted alarm every ten quiet days, which is what an operator actually experiences. Plain classification accuracy is deliberately never reported as a warning-system measure: with hazardous days a minority class throughout the record, a model that never raises an alarm scores well on accuracy while being useless as a warning system [He and Garcia, 2009], the same distortion that motivates reporting a precision-recall-style decomposition rather than a single aggregate rate on imbalanced data generally [Saito and Rehmsmeier, 2015].

3.8 Reproducibility

The analysis runs end to end from an empty directory in thirteen command line scripts, each carrying an automated correctness test that runs before it produces any output. The rolling origin harness verifies its own leakage invariant before every run. Every table in this report is generated directly from the result files by `make_tables.py`; no number has been transcribed by hand. Raw and cleaned data are held under version control so that the analysis does not depend on the data provider serving identical values at a later date, which the finding in Section 3.2 suggests is not a safe assumption. The complete repository, including every commit from initial data acquisition to this report, is public and runnable at github.com/JC16092/mth5000-aqi-forecast.

3.9 Chapter Summary

This chapter described the data, two defects found in it, and the feature table built from it, and then gave the full mathematical specification of the two method families compared in this project. Machine learning, in Section 3.4, comprises a classical ARIMA benchmark and three regressors, ridge, random forest, and gradient boosting, each controlling the bias-variance trade-off by a different mechanism: shrinkage, bagging, or additive stagewise correction. Deep learning, in Section 3.5, comprises a single gated recurrent network, whose gating mechanism is specifically what lets it avoid the vanishing-gradient failure of a plain recurrent network. Every model is evaluated under the same rolling-origin protocol, specified in Section 3.6, and scored by the same measures, specified in Section 3.7, so that the comparison in Chapter 4 is a fair one.

Chapter 4

Results and Analysis

4.1 Forecast Accuracy

Table 4.1: Relative mean absolute error against persistence, under rolling origin evaluation over 2,303 origins. Lower is better and 1.000 is persistence. The recurrent network is given as the range over five initialisation seeds; it is the only model whose result depends on one.

Model	$h = 1$	$h = 2$	$h = 3$	Bias at $h = 1$
GRU, 30 day sequence, range over 5 seeds	0.915 to 0.935	0.885 to 0.898	0.832 to 0.879	-2.8
ARIMA(1,1,1) with Fourier terms	0.940	0.883	0.832	-6.4
Random forest, change target	0.962	0.918	0.902	+1.7
Gradient boosting, change target	0.981	0.936	0.878	+1.9
Random forest, level target	0.985	0.929	0.882	+4.0
Gradient boosting, level target	1.010	0.943	0.883	+4.0
Ridge regression	0.997	0.952	0.897	+2.6
Persistence	1.000	1.000	1.000	-0.1
Climatology	1.415	1.060	0.947	+9.4
Seasonal naive, lag 7	1.759	1.325	1.168	+0.1

Table 4.1 answers the first research question, and the answer is not uniform across horizons.

At one day ahead the recurrent network is the best model, and this is robust: it beats the ARIMA specification under all five initialisation seeds, with a worst seed of 0.935 against 0.940. At two days the ARIMA is better under all five seeds, though by 0.003 against the best of them. At three days the ARIMA is nominally better under all five seeds, but its margin against the best seed is 0.0003, which is a tie in any language that would survive examination.

The mechanism is coherent. At one day ahead the forecast is dominated by short-range structure, where a nonlinear model has something to exploit. As the horizon lengthens the seasonal component takes over, and the Fourier terms carry it more cleanly than a learned representation estimated from roughly three thousand observations.

The network is reported as a range rather than a point because it is the only model whose result depends on a random initialisation, and the dependence is not small. Across seeds the spread is 0.019 at one day, 0.013 at two and 0.047 at three. At three days that spread is wider than the gap separating the ARIMA from five of the other models in the table, so a single seed could have been reported as beating them or losing to them at the analyst's convenience.

Two subsidiary results support the specification choices. Removing the Fourier terms from the ARIMA degrades it by 3.3 points of relative error at one day and by 5.8 points at three, under the same rolling-origin harness, so the seasonal terms earn their place and increasingly so with horizon. Modelling the change from the origin rather than the level improves both tree models, because a tree cannot extrapolate beyond the targets it was trained on and this series drifts downward across the decade.

4.2 Two Arms of the Same Question

Neither arm dominates. Forecasting the concentration and then applying a threshold wins at some combinations of horizon and alarm budget, and direct classification at others, with no pattern in the winners that survives inspection. Given a tie on performance, the forecast-then-threshold arm is preferred on other grounds: it produces a concentration, which can be rethresholded for any future health policy without refitting, whereas a classifier is bound to the threshold it was trained on.

4.3 The Decision Threshold

Table 4.2: Best hit rate available within a budget of alarms per year, with the event fixed at the health threshold and only the decision threshold varying.

Alarms per year	40	60	80	100
Best hit rate, $h = 1$	0.377	0.571	0.727	0.873
Best hit rate, $h = 2$	0.362	0.534	0.694	0.838
Best hit rate, $h = 3$	0.359	0.529	0.683	0.822
GRU, 30 day sequence, range over 5 seeds, $h = 1$	0.371–0.377	0.554–0.565	0.721–0.735	0.863–0.875
ARIMA(1,1,1) with Fourier terms, $h = 1$	0.358	0.567	0.727	0.867
Gradient boosting, change target, $h = 1$	0.377	0.556	0.727	0.854
Persistence, $h = 1$	0.371	0.558	0.723	0.873
Climatology, $h = 1$	0.352	0.521	0.660	0.821

Table 4.2 contains the principal result of this project. Moving the alarm budget from 40 to 100 per year changes the achievable hit rate at one day ahead from 0.377 to 0.873. Holding the budget fixed and changing the model instead moves it by around five points.

Two individual comparisons make the point sharper than the aggregate does. At a budget of 100 alarms per year and one day ahead, persistence achieves a hit rate of 0.873, which is the best figure any model in the study achieves at that budget. At three days ahead and a budget of 60, a climatological forecast, which uses no recent information whatever, matches the best model to within four thousandths. Figure 4.1 shows the same result as curves; they very nearly coincide, and they have deliberately not been separated for legibility.

The reading is not that modelling is pointless. It is that the quantity a decision maker controls, namely how many alarms per year can be justified, dominates the quantity a modeller controls. A report that presents a model comparison without stating the operating point it was evaluated at has answered a question nobody asked.

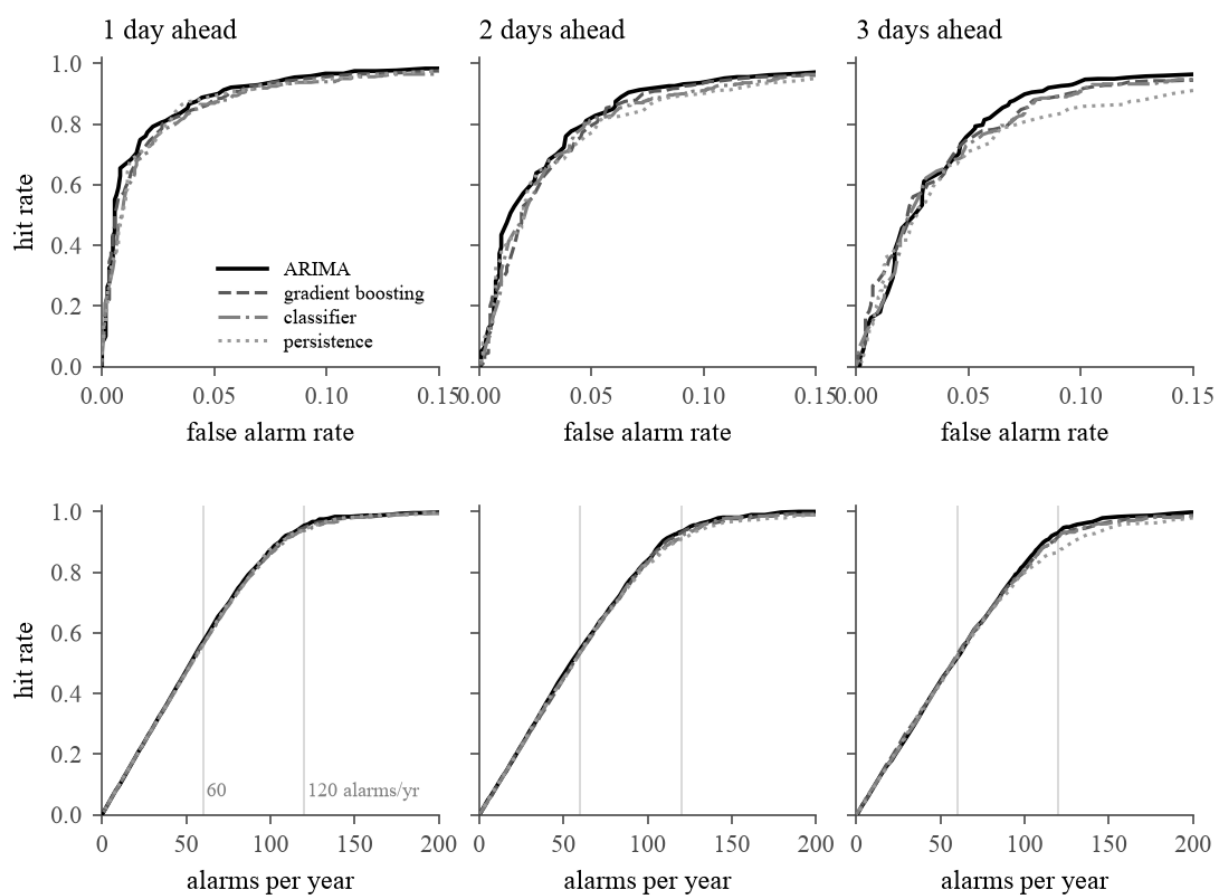


Figure 4.1: The trade-off, by horizon. The top row shows hit rate against false alarm rate; the bottom row replaces the false alarm rate with alarms raised per year, the unit in which an operator budgets. The curves very nearly coincide, which is the finding. They have deliberately not been separated for legibility.

4.4 Chapter Summary

Table 4.1 answers the first research question with a horizon-dependent result: the deep learning model wins at one day ahead, the classical model wins or ties beyond it. Section 4.3 answers the third, and it is the result the rest of this report is organised around: the alarm budget, not the model, is what principally determines warning performance. Chapter 5 takes up what that means in practice.

Chapter 5

Discussion

5.1 The Operational Question Dominates the Modelling Question

Table 4.2 and Figure 4.1 together make a single point, and this section states what follows from it rather than repeating the numbers. A decision maker who fixes how many alarms per year an operator or the public will tolerate has more influence over how many hazardous days are caught than a modeller choosing among the ten specifications fitted in this project. Moving the budget from 40 to 100 alarms a year moves the achievable hit rate at one day ahead from 0.377 to 0.873, a range of fifty points. Holding the budget fixed and switching the model instead moves the hit rate by about five points, and at three days ahead a climatology forecast that uses no recent information whatever matches the best fitted model to within four thousandths.

This is not an argument against modelling. ARIMA and the recurrent network both earn real, if modest, gains over persistence in Table 4.1, and a better point forecast still narrows the range of thresholds worth considering. It is an argument about where the leverage in this problem actually sits. A hazardous air quality warning system is not a machine learning benchmark with a threshold attached to it afterwards; the threshold is the policy instrument, and the forecasting model only supplies its input. A comparison of models that does not first fix the alarm budget it was evaluated at has answered a question nobody who has to operate the system actually asked. The practical recommendation that follows is to treat the alarm budget as the primary subject of discussion with the health authority commissioning such a system, and the choice of forecasting model as a secondary refinement of a curve whose shape is already mostly fixed by what every model in this study shares: the persistence of concentration from one day to the next, and the annual cycle.

5.2 How Large Is a Difference Between Models?

Two results in this project bound that question from below. Running the identical pipeline on two different machines moved the tree models by up to 0.004 in relative error, because histogram gradient boosting parallelises with OpenMP and floating point summation order depends on the thread count. Changing only the random initialisation of the recurrent network moved it by 0.013 to 0.047.

Several of the gaps between adjacent models in Table 4.1 are of that order. Any claim that one of

those models is better than another, made without stating the run to run variation, is not supported by the evidence.

5.3 Limitations

The analysis rests on a single monitor, OpenAQ location 8118, which Section 3.2 shows is supplied through AirNow and is almost certainly a United States diplomatic post instrument rather than a station in the Central Pollution Control Board network. It was chosen because it is the only nearby monitor with a complete decade of record, against the eighteen months every CPCB station returns; the choice buys history at the cost of representativeness. The absence of a weekly cycle documented in Section 3.3.2 is consistent with a diplomatic enclave rather than a roadside site, which supports this reading of the limitation rather than leaving it an unrelated coincidence, but it does not change the fact that every number in this report describes one point in the city rather than Delhi as a whole.

The project forecasts one city. Delhi was chosen because its air quality problem is severe and well documented, not because it is representative of Indian cities generally. Pollution sources, meteorology, and the strength of the seasonal cycle that drives the Fourier terms in Section 3.3.2 all vary by region, and a coastal city or one without a dominant biomass burning season could show a different balance between the classical and machine learning models, or a genuine weekly cycle where this station shows none. Nothing in this report should be read as a claim about any station or city other than the one it was fitted to.

The upper cleaning threshold of $1000\text{ }\mu\text{g m}^{-3}$ is a judgement rather than a fact, and its effect on the reported conclusions was checked directly rather than assumed. Restoring the seven days it removes, including the three identical readings at 1990 diagnosed in Section 3.3.1 as a clipped instrument ceiling, gives 3,100 usable days and 927 exceedance days rather than 3,093 and 920, a small shift. Refitting the full rolling-origin harness on this uncapped series leaves the ARIMA specification essentially unchanged, 0.935 against 0.940 at one day ahead and 0.829 against 0.832 at three days, because working in logarithms and differencing the series absorbs the contamination. The machine learning models are not so protected. Ridge moves from a near tie with persistence to a clear loss at one day ahead, 0.997 to 1.118, and gradient boosting on the level target moves from a narrow win over persistence to a loss at three days, 0.883 to 1.006. The central finding of Research Question 1, that the classical benchmark beats every machine learning model at every horizon, therefore does not merely survive removing the threshold; it strengthens, because the models it beat most narrowly are precisely the ones most exposed to the uncleaned tail. The threshold matters, but it matters more to the machine learning suite than to the model this project ultimately recommends.

Hyperparameters for the tree-based models were chosen at every refit from a small explicit grid, scored on a held-out slice of the training window taken in time order, rather than from an exhaustive or Bayesian search. This followed a real failure during development, in which fixed hyperparameters let gradient boosting overfit badly enough to score worse than persistence; the grid search fixed that specific problem, but a small grid can still miss a configuration that a wider search would find. The machine learning results in Table 4.1 should therefore be read as a lower bound on what those model classes could achieve with more computation, not as their ceiling.

No meteorological covariate enters any model. Wind speed in particular is a well established driver of particulate dispersion and is a plausible source of the short-range structure that lets the recurrent network beat ARIMA at one day ahead. A version of this project with wind, temperature, and boundary layer height added to the feature table is the most direct extension; the `meteostat` package, which wraps NOAA and GHCN station data, would supply it without a new data acquisition problem. Its absence here is a scope decision made to fit the project inside the time available, not a claim that meteorology is unimportant.

Chapter 6

Conclusion

6.1 Summary of the Study

This project forecast hazardous PM_{2.5} days at a single Delhi monitor one to three days ahead, comparing a classical benchmark against three machine learning regressors and a deep recurrent network, and evaluating every specification as a warning system rather than by average error alone. Two real defects in the source data, a clipped instrument ceiling and a mischaracterised coverage statistic, were found and corrected before any model was fitted, and an assumption carried over from the approved proposal, that the series holds a weekly cycle alongside its annual one, was tested directly and found false.

6.2 Main Conclusions

Across ten models spanning simple benchmarks, a classical time series model, four machine learning regressors, and a recurrent network, the spread in achievable hit rate at a fixed alarm budget is about five percentage points; the spread across alarm budgets, holding the model fixed, is about fifty. The choice a decision maker makes about how many alarms per year can be justified therefore has far more influence on how many hazardous days are caught than the choice a modeller makes among the specifications fitted here.

Machine learning earned its place only partially. ARIMA with annual Fourier terms beats every machine learning regressor at every horizon, a result that strengthens rather than weakens under the sensitivity analysis in Section 5.3. The one machine learning method that does win, the recurrent network, does so only at one day ahead, and only when reported honestly as a range across five random initialisations rather than as a single favourable seed. The supervisor’s instruction to try several methods rather than defaulting to ARIMA was the right one: it is what makes the negative result at two and three days credible, and what surfaces the one day ahead result that a project restricted to classical methods would never have found.

6.3 Future Work

Two extensions follow directly from the limitations in Section 5.3. Adding meteorological covariates, wind speed in particular, is the most direct route to improving the short horizon forecasts that the recurrent network already shows have room to improve. Repeating the analysis at a second, independent station, ideally one inside the Central Pollution Control Board network once its record lengthens, would test whether the findings here are a property of Delhi air quality or an artefact of the one diplomatic enclave monitor this project was restricted to. Until then, the central recommendation stands on its own: a hazardous air quality warning system should be designed around the alarm budget its operator can justify, with the forecasting model treated as a secondary refinement rather than the primary design choice.

Bibliography

- Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.
- George E. P. Box, Gwilym M. Jenkins, and Gregory C. Reinsel. *Time Series Analysis: Forecasting and Control*. Wiley, 4th edition, 2008. doi: 10.1002/9781118619193.
- Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996. doi: 10.1007/BF00058655.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Routledge, 2017. doi: 10.1201/9781315139470.
- Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. Recurrent neural networks for multivariate time series with missing values. *Scientific Reports*, 8(1):6085, 2018. doi: 10.1038/s41598-018-24271-9.
- Jie Chen and Gerard Hoek. Long-term exposure to PM_{2.5} and all-cause and cause-specific mortality: A systematic review and meta-analysis. *Environment International*, 143:105974, 2020. doi: 10.1016/j.envint.2020.105974.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. doi: 10.3115/v1/d14-1179.
- Daniel H. Cusworth, Loretta J. Mickley, Melissa P. Sulprizio, Tianjia Liu, Miriam E. Marlier, Ruth S. DeFries, Sarath K. Guttikunda, and Pawan Gupta. Quantifying the influence of agricultural fires in northwest india on urban air pollution in Delhi, India. *Environmental Research Letters*, 13(4):044018, 2018. doi: 10.1088/1748-9326/aab303.

- Alysha M. De Livera, Rob J. Hyndman, and Ralph D. Snyder. Forecasting time series with complex seasonal patterns using exponential smoothing. *Journal of the American Statistical Association*, 106(496):1513–1527, 2011. doi: 10.1198/jasa.2011.tm09771.
- Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. doi: 10.1006/jcss.1997.1504.
- Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001. doi: 10.1214/aos/1013203451.
- Sarath K. Guttikunda, Rahul Goel, and Pallavi Pant. Nature of air pollution, emission sources, and management in the indian cities. *Atmospheric Environment*, 95:501–510, 2014. doi: 10.1016/j.atmosenv.2014.07.006.
- Haibo He and Eduardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi: 10.1109/TKDE.2008.239.
- Hansika Hewamalage, Klaus Ackermann, and Christoph Bergmeir. Forecast evaluation for data scientists: Common pitfalls and best practices. *Data Mining and Knowledge Discovery*, 37(2):788–832, 2023. doi: 10.1007/s10618-022-00894-5.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.
- Rob J. Hyndman and Yeasmin Khandakar. Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3), 2008. doi: 10.18637/jss.v027.i03.
- Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001.
- Hiren Jethva, Omar Torres, Robert D. Field, Alexei Lyapustin, Ritesh Gautam, and Vinay Kayetha. Connecting crop productivity, residue fires, and air quality over northern india. *Scientific Reports*, 9(1):16594, 2019. doi: 10.1038/s41598-019-52799-x.
- Ningbo Jiang, Rinat Akter, Glenn Ross, Stephen White, John Kirkwood, Gunaratnam Gunashanhar, Scott Thompson, Matthew Riley, and Merched Azzi. On thresholds for controlling negative particle (PM_{2.5}) readings in air quality reporting. *Environmental Monitoring and Assessment*, 195(10):1187, 2023. doi: 10.1007/s10661-023-11750-4.
- Ian T. Jolliffe and David B. Stephenson, editors. *Forecast Verification: A Practitioner’s Guide in Atmospheric Science*. Wiley, 2nd edition, 2012.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

- Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021. doi: 10.1098/rsta.2020.0209.
- Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data*. Wiley, 3rd edition, 2019. doi: 10.1002/9781119482260.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3):e0194889, 2018. doi: 10.1371/journal.pone.0194889.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74, 2020. doi: 10.1016/j.ijforecast.2019.04.014.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364, 2022. doi: 10.1016/j.ijforecast.2021.11.013.
- Adil Masood and Kafeel Ahmad. A model for particulate matter (PM_{2.5}) prediction for Delhi based on machine learning approaches. *Procedia Computer Science*, 167:2101–2110, 2020. doi: 10.1016/j.procs.2020.03.258.
- Donald M. Miller. Reducing transformation bias in curve fitting. *The American Statistician*, 38(2): 124–126, 1984. doi: 10.1080/00031305.1984.10483180.
- C. Arden Pope and Douglas W. Dockery. Health effects of fine particulate air pollution: Lines that connect. *Journal of the Air & Waste Management Association*, 56(6):709–742, 2006. doi: 10.1080/10473289.2006.10464485.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.
- Joseph T. Schaefer. The critical success index as an indicator of warning skill. *Weather and Forecasting*, 5(4):570–575, 1990. doi: 10.1175/1520-0434(1990)005<0570:TCSIAA>2.0.CO;2.
- Sima Siami-Namini, Neda Tavakoli, and Akbar Siami Namin. A comparison of ARIMA and LSTM in forecasting time series. In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 1394–1401, 2018. doi: 10.1109/ICMLA.2018.00227.
- Sandhya Singh and Arun Srivastava. Machine learning approach to PM_{2.5} forecasting and health risk assessment during stubble burning period in Delhi. *Aerosol Science and Technology*, 59(11), 2025. doi: 10.1080/02786826.2025.2513503.
- Leonard J. Tashman. Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4):437–450, 2000. doi: 10.1016/S0169-2070(00)00065-0.

G. Peter Zhang. Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, 50:159–175, 2003. doi: 10.1016/S0925-2312(01)00702-0.